

Ask: Allow Only One Mailing Per Revision

Document Number: P4169R0
Date: 2026-03-29
Audience: WG21
Reply-to: Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: April 2026 (post-Croydon mailing)

1. Disclosure

2. Why I Rise

2.1. My Dilemma

2.2. My Anguish

3. One Mailing, One Revision

4. Prior Art: P2138R4

4.1. P2138R4 Identified This Problem Five Years Ago

4.2. From Judgment to Bright Line

4.3. The Consequence of Forgetting History

4.4. Eighteen Implementers Asked for This

5. The Croydon Evidence

5.1. Design Changes That Were Never in Any Mailing

5.2. Papers That Could Not Be Read in Isolation

5.3. Wording Bug Fixes Are Not the Problem

5.4. This Is Not About Croydon

6. The Rule's Circular Paradox

6.1. Feature Removals Would Be Forbidden

6.2. Allow Bug Fixes, or Get Deadlocked

6.3. Three Ways Out

7. The Structural Incentive Problem

8. Not Ready for Mailing Means Not Ready for Standard

9. Why Bright-Line Rules Work Better Here

10. All the Objections, Answered

“We have always done in-meeting revisions”

“LWG reviewed it in-room”

“We were under deadline pressure”

“The author’s competing proposals explain the paper”

“The changes were editorial”

“You are proposing to slow down the committee”

“National body delegates in the room saw the changes”

“The rule would have killed C++26”

“Evaluate in-meeting revisions case by case”

“The rule can be used to filibuster a paper”

“The rule would gridlock NB comment resolution”

11. Proposed Amendment to SD-4

Acknowledgements

References

Abstract

This paper asks that revisions to most papers be disallowed during meetings, to prevent delegates from voting on wording they have not yet reviewed.

At Croydon, multiple papers were revised during the meeting week and voted on at Saturday plenary in versions that never appeared in any pre-meeting mailing. The revisions removed public APIs, narrowed design options, and cross-referenced each other - forming a web of normative changes that existed as a coherent whole only during the meeting itself. The pattern is not unique to Croydon or to any particular feature area; the structural pressure to revise in-room intensifies near the end of every release cycle. This paper proposes an amendment to SD-4: a paper seeking a plenary motion with normative wording changes must have appeared, in the revision to be voted on, in a pre-meeting mailing. In-meeting revisions that change normative design restart the clock. Wording bug fixes are exempt. The principle is simple: one mailing, one revision.

Revision History

R0: April 2026 (post-Croydon mailing)

- Initial version.
-

1. Disclosure

The author provides information and serves at the pleasure of the committee.

The author is the founder of the C++ Alliance. The author maintains competing proposals in the

`std::execution` space: P4003R0^[1], P4007R0^[2], P2583R0^[3], and P4100R0^[4], “The Network Endeavor.” This paper proposes a process rule that would apply to every paper in every feature area, including the author’s own. The author’s preferred asynchronous model competes with `std::execution`. The reader should calibrate everything that follows accordingly.

The proposed rule, if it had been in effect at Croydon, would also have prevented the author from submitting last-minute normative revisions to his own papers. The author accepts that constraint.

This paper is a companion to P4131R0^[5] (the train model), P4129R1^[6] (voting dynamics), and P4130R0^[7] (appointment is policy). Those papers examine the schedule, the mechanism, and the officer. This paper examines the mailing.

2. Why I Rise

I care about the standard more than I care about my position in the room. Each of those two costs me something.

2.1. My Dilemma

I have competing papers. I understand why a reader would see this paper as an attack on the authors whose work competes with mine.

The evidence in this paper draws heavily from `std::execution` papers. `std::execution` is the largest feature in C++26 - hundreds of pages of specification touching concurrency, type erasure, customization points, coroutine integration, and allocator models. A feature of that scale generates more in-meeting revisions, more

cross-dependencies, and more normative surface area than any other feature in the release. The proposed rule would affect `std::execution` most, because it is the feature most exposed to the problem the rule addresses.

If I wanted `std::execution` to fail, I would want the opposite of what I am proposing. I would want an environment where last-minute changes go unchecked, where mistakes accumulate under pressure, where normative text is written in a conference room on a Tuesday and voted on Saturday. That environment maximizes defect probability. The rule I propose reduces it.

The authors of the in-meeting revisions documented in Section 5 did what was locally rational at every step. They invested years in `std::execution`. When LWG review surfaced issues at the final meeting, the choice was: fix it now or ship it broken. That is not a real choice. The system left them nothing else. At each mailing, at each meeting, each author did what they genuinely believed was best for C++. Each step was individually reasonable. Globally, the sequence produced a Saturday plenary vote on normative text that no delegate had reviewed in its final form. I would likely have done the same thing in their position. Most authors would.

The authors' intentions were good. Their skill is not in question. Their experience is not in question. The problem is that no amount of good intention, skill, or experience changes the structural risk of making normative design changes at the last meeting of a three-year cycle under that kind of pressure. The tolerance for mistakes in the International Standard is zero, and the process should not put any author - however talented - in a position where the only responsible choice is to bypass the review chain.

The authors did not make poor choices. They were given poor choices.

2.2. My Anguish

I prepared for Croydon the way the process asks every delegate to prepare. I read the mailing. I studied the revisions. I cross-referenced the wording. I identified concerns. I came to the meeting ready to participate in the review of the text the national body review chain had seen.

Then during the meeting week, the text changed. APIs were removed. Design options were narrowed from three to one. New papers appeared that had never been in any mailing. The text I had prepared for was not the text being voted on Saturday. My preparation was not wasted - I had learned the domain - but the specific arguments I had assembled no longer applied to the version in front of the room.

I watched design compromises get locked into the standard under time pressure. Choices that might have gone differently with one more mailing cycle of review. Choices that will be frozen into the ABI. I was a new delegate. The process did not give me a way to intervene that would not mark me as the person who delayed the room. So I sat in plenary, and I watched, and I said nothing.

Plenary is a ratification step. Substantive concerns belong in the working group sessions or on the reflector - I understand that now. But nobody explained it to me in those terms. What I was told, by two different people, was: do not object to unanimous consent, or everybody will resent you for making people miss their flights. That

is not a principled argument about where concerns belong. That is a social warning. The process has a right answer for where I should have raised my concerns. The guidance I actually received had nothing to do with that answer.

The Standard Ships for a Decade

The specification text voted on that Saturday will be read by developers for a decade. The design compromises made under time pressure during that meeting week - choices rushed through in versions no national body expert had reviewed - will be frozen into implementations, into ABIs, into codebases that depend on the standard being right. One more mailing cycle of review might have produced different choices. It might not have. The point is that the review never happened.

Every compromise in `std::execution` that ships into the standard makes my competing proposals look better. I should want those compromises to ship. I do not. I want C++ to win, even when winning means I lose. That is my anguish.

3. One Mailing, One Revision

This paper proposes the following amendment to SD-4^[8]:

A paper seeking a plenary motion that applies normative changes to the working draft must have appeared, in the revision to be voted on, in a pre-meeting mailing published before the face-to-face meeting at which the plenary vote occurs.

If LWG or CWG review during the meeting identifies issues requiring normative changes to the mailed revision, the paper returns to the next pre-meeting mailing with a new revision number. The revised paper may be voted on at the subsequent face-to-face meeting.

Two exceptions:

- Editorial changes - formatting, cross-references, stable-name adjustments, and typographical corrections - do not restart the clock.
- Defect reports processed through the existing LWG/CWG issue machinery are not affected. They have their own review chain.

The rule applies only to papers seeking plenary motions with normative wording changes at face-to-face meetings. It does not affect design papers, direction papers, issue papers, or followup/rebuttal papers, which SD-4 already permits as late submissions. Telecon-driven design iteration between face-to-face meetings is unaffected - the rule governs only the version that reaches plenary.

SD-4 currently states that followup papers to on-time papers can be considered, and that late papers without an on-time predecessor are not actionable. But SD-4 does not prohibit in-meeting normative wording revisions from reaching plenary. This paper proposes closing that gap.

4. Prior Art: P2138R4

The committee considered this problem five years ago. A remedy was proposed, voted on, and fell short of consensus. The problem it addressed persisted.

4.1. P2138R4 Identified This Problem Five Years Ago

In 2021, Ville Voutilainen proposed P2138R4^[9], “Rules of Design<=>Specification engagement.” The paper addressed a structural problem in the committee’s process: normative wording was reaching plenary without adequate review. The paper proposed a “Tentatively Plenary” state - after specification review, a paper waits until the next meeting for plenary vote by default - along with implementation and deployment experience review before plenary.

Peter Dimov stated the problem P2138R4 was designed to solve^[9]:

Roughly and approximately, the problem that the committee can potentially inflict billions of dollars of damage on the C++ community by breaking code in a sufficiently problematic way. This requires a certain amount of due diligence.

The LEWG poll to adopt P2138R4 as official process failed to reach consensus^[10]:

SF	WF	N	WA	SA
5	14	2	6	6

Nineteen delegates favored adoption. Twelve opposed. The direction had majority support but not consensus. The objections were substantive and offered in good faith: concerns about gatekeeping, discouraging participation, and slowing the process. The committee concluded that P2138R4’s mechanism was too heavy for

the problem it addressed.

Five years later, at Croydon, the problem recurred. This paper proposes a lighter mechanism.

4.2. From Judgment to Bright Line

The author of this paper did not know P2138R4 existed until Voutilainen mentioned it in private correspondence. That discovery made this paper better. Voutilainen drew the same line this paper draws (quoted with permission):

I trust your goal is to avoid seeing an R7 when all you were able to read beforehand was an R2. And especially seeing a design you've never heard of, when R2 had a completely different design.

The core insight of P2138R4 - that normative wording needs a cooling period before plenary - is preserved in this paper. The mechanism is different. P2138R4's "Tentatively Plenary" bypass required an explicit, minuted decision by both the design group and the specification group - two judgment calls by two chairs. That judgment-heavy mechanism was a weakness. The proposed rule in this paper replaces it with a bright-line test: is the presented revision in the mailing? Yes or no. Section 9 explains why a bright-line rule works better here.

This paper is P2138R4's descendant, refined by the lesson that structural simplicity succeeds where judgment-dependent complexity does not.

4.3. The Consequence of Forgetting History

The committee's prior art is how the committee learns. P2138R4 sat in the mailing for five years. The author of this paper did not know it existed until Voutilainen mentioned it. Presumably most delegates at Croydon were similarly unaware. The problem P2138R4 was designed to prevent recurred - not because anyone chose to ignore the paper, but because the committee has no systematic way to surface its own prior art when it is most relevant.

These artifacts - papers, polls, rationales, the reasoning behind rejected proposals - are how institutional memory works. Preserving, indexing, and surfacing them before consequential decisions would help the committee build on its own prior work rather than rediscovering problems independently. The author is grateful to Voutilainen for pointing to P2138R4. Every committee member who writes a process proposal - including this author - benefits from searching the mailing for the proposals that came before.

4.4. Eighteen Implementers Asked for This

Meanwhile, P3962R0^[11], “Implementation reality of WG21 standardization,” arrived in the same mailing cycle. Nina Ranns and 17 implementer co-authors reported^[11]:

Full conformance to recent standards remains difficult in practice, with some implementations still working toward C++20 conformance with limited capacity to adopt newer standards.

And^[11]:

Implementation feedback is often introduced late, treated as adversarial, or framed primarily as an obstacle to progress rather than as essential design input.

And^[11]:

We would like the committee to consider ways of slowing down the addition of features into the standard to allow implementers to catch up.

The people who build the standard are asking for the kind of discipline P2138R4 proposed in 2021. This paper proposes a narrower, less disruptive version of that proposal, aligned with what the implementers are requesting. One mailing, one revision. The minimum viable discipline.

5. The Croydon Evidence

In-meeting revisions that fix wording bugs found during LWG or CWG review are reasonable and necessary. The alternative - pulling the entire feature because LWG found a typo - would itself require a removal paper, which is itself an in-meeting normative revision (Section 6 examines this paradox in detail). This section documents revisions at Croydon that appear to have made normative design changes - removing APIs, narrowing design options, changing the public interface - in versions that were never in any pre-meeting mailing. The author acknowledges that reasonable people may classify some of these changes differently. The per-paper entries in Section 5.1 address each case individually.

5.1. Design Changes That Were Never in Any Mailing

Each entry below states the factual record, then acknowledges the most reasonable defense of the revision. The reader can judge.

P3941R3^[12] - Scheduler Affinity (Dietmar Kühl). **P3941R2**^[13] appeared in the pre-Croydon mailing (2026-02-23). R3 appeared only on isocpp.org, not in any mailing. R3 removes `change_coroutine_scheduler` from the working draft. R3 was the version voted on at Croydon plenary, Saturday March 28. R2 was the last version any national body expert could have reviewed before the meeting.

Kühl is one of the most careful authors in the committee. The removal of `change_coroutine_scheduler` likely reflected a working group direction, not a unilateral decision. LWG reviewed the revision in-room. It is entirely possible that the R2-to-R3 delta was the right engineering call. None of that changes the structural fact: deleting a public API is a design change, and the version containing that deletion was never in any mailing.

P3980R1^[14] - Task's Allocator Use (Dietmar Kühl). **P3980R0**^[15] appeared in the pre-Croydon mailing (2026-02-22) and presented three wording options: A, B, and C. R1 appeared only on isocpp.org. R1 drops options B and C.

The working group may well have discussed all three options, reached consensus on option A, and directed Kühl to produce a clean revision. That is the normal and healthy output of working group review. But the national body experts who read the mailing saw three options. The version they could have commented on presented a choice. The version voted on Saturday presented a conclusion. Narrowing a design from three choices to one is a design decision, even when it is the right one.

P4159R0^[16] - Make `sender_in` and `receiver_of` exposition-only. This paper never appeared in any mailing. It was born at Croydon. It removes `sender_in` and `receiver_of` from the public interface by making them exposition-only.

Making concepts exposition-only could reasonably be classified as an interface simplification rather than a design change - the underlying constraints still apply, only the names become non-normative. A reasonable person could see this as editorial cleanup. The author of this paper acknowledges that the classification is debatable. What is not debatable is that the paper existed in no mailing at all. Even if the change is editorial in character, a paper with no mailing history has had zero days of national body review in any form.

P3826R5^[17] - Fix Sender Algorithm Customization (Eric Niebler). **P3826R3**^[18] appeared in the 2026-01 mailing - the last version available for national body review. R4 and R5 appeared only on isocpp.org - two full revisions past the mailed version. The R5 changelog (dated 2026-03-25, a Tuesday at Croydon) states: "For consistency with P3941R3, remove the two uses of the `write_env` algorithm."

Niebler has been iterating on sender algorithm customization for years. The removal of `write_env` in R5 may have been a mechanical consequence of the working group's decision in P3941R3 - not a new design choice in P3826 itself, but a downstream cleanup required for consistency. LWG reviewed the result. It is possible that

every individual change in R5 was reasonable, well-motivated, and technically correct. The concern is not the quality of the work. The concern is that this revision cross-depends on another in-meeting revision, and the version voted on Saturday was two full revisions past the last mailed version. Two revisions of unmailed iteration is two revisions that the national body review chain did not see.

5.2. Papers That Could Not Be Read in Isolation

The in-meeting revisions reference each other:

- P3826R5 depends on P3941R3 (removes `write_env` for consistency with R3)
- P3927R1^[19] (task_scheduler bulk) rebases wording on P3941R3
- P4154R0^[20] (Renaming execution things) depends on P3826R5

Cross-dependencies among papers are normal. `std::execution` is the largest feature in C++26 - hundreds of pages of specification. When LWG review finds an issue in one paper, the fix naturally propagates to papers that share wording or depend on the same definitions. The authors were doing what authors do: keeping the specification internally consistent.

The concern is not that the papers reference each other. The concern is that no single paper could be reviewed in isolation, and the web of references existed only in versions that were never in any mailing. A delegate who wished to understand what plenary was voting on Saturday would need to read all of these simultaneously, during the meeting week, in versions that appeared on isocpp.org days before the vote.

The normative text of C++26 `std::execution` existed as a coherent whole only during the meeting itself. That may have been unavoidable given the feature's scale and the issues LWG discovered. This paper does not claim the authors acted in bad faith. It claims the process permitted an outcome that should not be permitted again.

5.3. Wording Bug Fixes Are Not the Problem

Other in-meeting revisions at Croydon - P3373R3, P3149R10, P3981R2, P3795R2, P3978R3 - were wording bug fixes from LWG review. They preserved the mailed design. They are not part of the concern this paper raises. Section 6 explains why.

5.4. This Is Not About Croydon

The author attended Croydon, and it is the meeting he can document from direct observation. The evidence in Section 5.1 draws from `std::execution` because `std::execution` was the largest feature at that meeting and generated the most in-meeting revisions.

But the pattern is almost certainly not unique to Croydon, and not unique to `std::execution`. Any feature of sufficient complexity, at any meeting near the end of a release cycle, faces the same structural pressure: LWG review discovers issues, the mailing deadline has passed, and the author's only responsible choice is to revise in-room. The author believes that a similar examination of the final meetings before C++23 and C++20 would reveal the same pattern in different feature areas, with different authors, under the same constraints. That examination could be the subject of a future paper.

The proposed rule is not a response to one meeting or one feature. It is a response to a structural incentive that recurs whenever the committee approaches a shipping deadline. Croydon is the example because the author was there. The rule is general because the problem is general.

6. The Rule's Circular Paradox

The proposed rule has a circular problem that must be stated honestly.

6.1. Feature Removals Would Be Forbidden

The train model says: if a feature is not ready, remove it. But removing a feature from the working draft requires normative wording - a paper with deletions, cross-reference fixups, and feature-test macro changes. Writing that removal paper during the meeting is itself an in-meeting normative revision. A strict "no in-meeting normative wording" rule would make the train model's safety valve inoperable at the final meeting.

6.2. Allow Bug Fixes, or Get Deadlocked

In-meeting revisions that fix wording bugs found during LWG or CWG review are reasonable and necessary. The alternative - pulling the entire feature because LWG found a typo in a Mandates clause - would require a removal paper, which is itself an in-meeting normative revision. Bug fixes must be permitted or the process deadlocks.

The proposed rule therefore applies only to normative design changes - not to wording corrections that preserve the mailed design. A wording correction preserves the mailed design when it does not add, remove, or rename any public-facing API; does not change observable behavior or semantics; and does not narrow or eliminate options presented in the mailed revision. Anything outside that boundary is a design change and restarts the clock.

The author of P2138R4, in private correspondence (quoted with permission), drew the same distinction:

We do need to update some papers. Ones that are doing wording fixes.

6.3. Three Ways Out

The paradox admits several solutions. The following are presented as options for the committee to contemplate.

Bug-fix-only final meeting. The last meeting before CD or DIS is restricted to wording bug fixes that preserve the mailed design. No new normative design changes. Features that still need design changes at that point miss the train. The circular problem does not arise because the feature either ships as-mailed or does not ship.

Poison pill papers. Every feature above a certain complexity threshold is accompanied by a companion paper containing the wording to remove it, submitted in the same mailing as the feature's wording. If the feature needs to be pulled, the removal wording already exists and has been through the mailing chain.

Standing removal authority. A standing editorial instruction allows plenary to vote "remove Section X.Y" without a paper, since removal is always a revert to a known prior state - the previous working draft. This is the lightest-weight option but may need CWG and LWG buy-in on the editorial mechanics.

7. The Structural Incentive Problem

Nobody sets out to bypass the mailing review chain. But when the process permits in-meeting normative revisions without constraint, it creates a structural asymmetry that nobody designed and nobody wants.

Authors who submit polished wording by the mailing deadline expose their text to the full national body review chain - weeks of scrutiny from domain experts, implementers, and users who may never attend a meeting. Authors whose wording is still evolving at the mailing deadline face a choice: submit incomplete text and iterate in-room, or miss the meeting cycle. Most authors - reasonably - choose the former. The result is that the most thoroughly prepared wording receives the most scrutiny, while wording that reaches its final form during the meeting week receives less. This is not anyone's intention. It is the structural consequence of allowing in-meeting normative revisions to reach plenary.

The asymmetry affects delegates too. A delegate who spent hours cross-referencing the mailed revision, identifying issues, and preparing arguments for the meeting may arrive to discover that a new revision appeared during the meeting week. The preparation is not wasted - the delegate learned the domain - but the specific arguments may no longer apply to the text being voted on. Over time, this creates a quiet disincentive to prepare thoroughly for mailed revisions. No delegate consciously decides to prepare less. The structure makes thorough preparation less reliably useful.

SD-4 states that "any design change made between the ballot and publication will be expected to have near-unanimous consent in subgroups and in plenary"^[8].

Near-unanimous consent from delegates who have not had the opportunity to review the final text in advance is structurally different from consent given after weeks of mailing review. The proposed rule closes that gap.

8. Not Ready for Mailing Means Not Ready for Standard

The train model (P1000R2^[21], “C++ IS Schedule,” through P1000R7^[22]) exists to eliminate deadline pressure. “Ship what’s ready, defer what’s not”^[21]. Under the train model, there is no such thing as deadline pressure. The train leaves on schedule. Features board when ready.

This is easy to say in the abstract and painful to apply in practice. When LWG review at the final meeting discovers a problem in wording that has been years in the making, the author faces an agonizing choice: revise now and bypass the mailing chain, or defer the fix and ship wording that everyone in the room knows is imperfect. Nobody wants to be the person who ships a known deficiency into the International Standard. The pressure to fix it now is real, human, and understandable.

But the train model’s answer to that pressure is clear. If a normative revision was not ready for the pre-meeting mailing, the train model says: ship with the mailed wording, or resolve the delta via defect report after the meeting. The feature still ships. The improvement waits one cycle.

The moment “we had to get this in before the train left” justifies bypassing the review chain, the train model is no longer operative. It becomes the old fixed-schedule model with feature rushing - the model the train was designed to replace. This is not a criticism of any author who faced that choice at Croydon. It is a criticism of a process that forces the choice at all.

If the wording was not ready for the mailing, it was not ready for the standard. P4131R0^[5] Section 2 documents the model’s testable claims.

9. Why Bright-Line Rules Work Better Here

A rule that requires the chair’s judgment creates three problems. First, outcomes depend on the chair’s skill, knowledge, and disposition - a single point of failure. A different chair applying the same rule may reach a different conclusion. Second, every judgment call is contestable. A delegate who disagrees with the chair’s assessment can challenge it, and the challenge has standing because the rule invited interpretation. Third, the volume of judgment calls is exhausting. A chair who must evaluate whether each in-meeting revision constitutes a normative design change or a wording bug fix - under time pressure, at the final meeting of a three-year cycle - carries a burden the process should not impose.

Rules with objective yes/no application eliminate all three problems. Outcomes are consistent regardless of who holds the chair. Decisions cannot be contested because there is nothing to contest. The chair’s workload drops because mechanical checks are fast.

P2138R4 (Section 4) did not reach consensus, and one contributing factor may have been that its mechanism was judgment-heavy. Bypassing Tentatively Plenary required an explicit, minuted decision by both the design group chair and the specification group chair - two judgment calls by two officers. The proposed rule in this paper requires nearly none: is the presented revision in the mailing? The only remaining question is whether a wording correction preserves the mailed design, and that question is itself bounded by a bright-line definition (Section 6.2): no API additions, no API removals or renames, no change in observable behavior, no narrowing of presented options. A working group chair can apply that checklist in the course of normal review without exercising discretion.

The proposed rule does not mandate quality. It does not tell authors to write better papers. It adjusts the structure so that the mailing deadline becomes the meaningful checkpoint for normative wording. Authors who need more iteration time simply take it - the next mailing cycle is always available. The structure makes thorough preparation the natural strategy, not because it punishes anyone, but because it aligns the process with the review chain the committee already depends on.

A well-designed process minimizes the number of decisions that require judgment. Every rule that can be converted from a judgment call to a checklist item makes the process fairer, less dependent on any individual, and less exhausting to administer. The committee would benefit from more rules like this one.

10. All the Objections, Answered

Eleven objections this paper anticipates, stated in their strongest form.

“We have always done in-meeting revisions”

They have, and often for good reason. In-meeting wording corrections from LWG review are a normal and valuable part of the process, and this paper does not propose to change that (Section 6.2). The question is narrower: should in-meeting revisions that make normative design changes - new APIs, removed APIs, narrowed options - reach plenary without having appeared in a mailing? The proposed rule draws that line. It does not prohibit in-meeting work. It requires that normative design changes go through the mailing review chain before plenary.

“LWG reviewed it in-room”

LWG review is real review. Its members are among the most careful readers in the committee, and their wording expertise is irreplaceable. Nothing in this paper diminishes that. The in-meeting revisions documented in Section 5 went through LWG, and the wording quality may well have improved as a result.

But LWG is a specification review group. Its review chain is not the same as the national body review chain, which includes domain experts, implementers, and users who read the mailing but do not attend the meeting. At Croydon, 130 delegates attended in person. The mailing reaches every national body expert in every member country. LWG review and mailing review serve different functions. The proposed rule does not question the quality of LWG review. It requires that the national body review chain also see the version being voted on.

“We were under deadline pressure”

The pressure was real. Authors who have spent years on a feature, who are sitting in the final meeting of a three-year cycle, who are told by LWG that the wording needs changes - those authors face genuine urgency, and it would be dishonest to pretend otherwise. But the train model exists precisely to eliminate that pressure (Section 8). If the process is working as designed, no author should ever be in a position where the choice is “revise now or lose the feature.” The feature ships on the mailed wording. The improvement comes next cycle.

“The author’s competing proposals explain the paper”

This is a fair concern, and the author takes it seriously. The conflict is disclosed in Section 1 precisely so that every reader can weigh the argument with full knowledge of the author’s position. The proposed rule would apply equally to the author’s own papers - and would have constrained his own ability to revise at Croydon had he submitted normative wording. The author accepts that constraint.

The rule is general. It applies to every paper in every feature area. The Croydon evidence in Section 5 draws from `std::execution` because `std::execution` was the largest feature at that meeting and generated the most in-meeting revisions - not because it competes with the author’s proposals. A similar examination of any final meeting in any release cycle would likely find the same pattern with different papers by different authors. The rule addresses a structural incentive, not a specific feature. The reader can and should calibrate accordingly.

“The changes were editorial”

Some of the in-meeting revisions documented in Section 5 are closer to editorial than others. Making concepts exposition-only (P4159R0) is a reasonable borderline case - Section 5.1 acknowledges this. Removing a public API (`change_coroutine_scheduler` in P3941R3) and narrowing three wording options to one (P3980R1) are harder to classify as editorial. The proposed rule does not require agreement on where every revision falls. It requires that the version voted on at plenary appeared in a mailing - a test that does not depend on classifying anything.

“You are proposing to slow down the committee”

P3962R0^[11] - signed by 18 implementers - asks the committee to slow down. “We would like the committee to consider ways of slowing down the addition of features into the standard to allow implementers to catch up”^[11]. This paper proposes a narrower version of that request.

“National body delegates in the room saw the changes”

Some did, and their review is valuable. But in-room review during a busy meeting week - while simultaneously attending sessions, reviewing other papers, and participating in evening discussions - is a different kind of review from the weeks of focused reading time that a pre-meeting mailing provides. The mailing also reaches every national body expert in every member country, including those who could not attend. Both kinds of review serve the process. The proposed rule ensures that the mailing review chain is not bypassed for normative design changes.

“The rule would have killed C++26”

Under the proposed rule, the committee would have had two options for each affected paper: vote on the mailed revision (R2 of P3941, R0 of P3980, R3 of P3826) or defer the delta to the next meeting. Neither option kills C++26. The first ships a version that the full national body review chain has seen. The second gives the improvement one more cycle. If the R2-to-R3 delta in P3941 was important enough to justify bypassing the review chain, it was important enough to survive one meeting cycle. If it was not important enough to survive one meeting cycle, it was not important enough to bypass the review chain.

“Evaluate in-meeting revisions case by case”

A case-by-case exception reintroduces the judgment the rule is designed to eliminate. If a chair has discretion to admit an in-meeting revision, the rule becomes a negotiation - and every author with an in-meeting revision has a reasonable argument for why their case deserves the exception. The chair is placed in an impossible position: evaluate each revision’s normative impact under time pressure, at the final meeting, with the political weight of years of committee work behind each paper. That is not a burden the process should impose on any chair. A bright-line rule removes the burden: is the presented revision in the mailing? Yes or no. Section 9 explains why a bright-line rule works better here.

“The rule can be used to filibuster a paper”

If the rule resets the clock on every normative design change, an opponent could - in theory - force design changes at every meeting, preventing the paper from ever reaching plenary. The filibuster has two modes, and the rule addresses both.

A bad-faith filibuster requires the objector to convince the working group that a design change is needed. If the room does not adopt the change, the clock does not reset. A failed motion is not a filibuster.

Honest complexity - where a feature is so large that real normative findings surface at every meeting - is a signal, not a bug. Where possible, the feature should be decomposed: ship the stable parts, extend later.

Decomposition does not always work. Large features have emergent properties, and individually unmotivated pieces may fail to gain consensus even when the whole would succeed. For features that genuinely cannot be decomposed, the group-boundary mechanism in Section 11 provides the answer.

CWG and LWG are specification groups. Their job is to translate a design into wording, not to change the design. If CWG or LWG determines during review that a design change is needed, the paper returns to EWG or LEWG - the groups whose job it is to make design decisions. That return resets the clock naturally. No special safety valve is needed because the group boundary is itself the bright line. CWG and LWG can make wording fixes that preserve the mailed design. Anything that requires a design decision goes back to evolution. Evolution means next mailing.

“The rule would gridlock NB comment resolution”

During the CD/DIS cycle, national bodies submit comments that the committee resolves at the next meeting. Those resolutions sometimes require normative design changes. Under the proposed rule, each such resolution would reset the clock to the next mailing, potentially stretching the CD/DIS cycle.

This is an open question. NB comment resolution operates under an external ISO deadline, not a self-imposed one. In theory, the group-boundary mechanism should provide sufficient flexibility: most NB comment resolutions that touch wording can be handled as wording corrections that preserve the design, and those that require design changes should go through the evolution groups regardless. In practice, the interaction between this rule and the CD/DIS timeline deserves careful consideration by the committee. This paper presents the question rather than prescribing an answer.

11. Proposed Amendment to SD-4

The following text is proposed as an amendment to SD-4^[8]:

Mailing discipline for normative wording papers. A paper seeking a plenary motion that applies normative changes to the working draft must have appeared, in the revision to be voted on, in a pre-meeting mailing published before the face-to-face meeting at which the plenary vote occurs. If LWG or CWG review during the meeting identifies issues requiring normative changes to the mailed revision, the paper returns to the next pre-meeting mailing with a new revision number. The revised paper may be voted on at the subsequent face-to-face meeting. Telecon-driven design iteration between face-to-face meetings is unaffected.

Exceptions. Editorial changes - formatting, cross-references, stable-name adjustments, and typographical corrections - do not restart the clock. Defect reports processed through the existing LWG/CWG issue machinery are not affected; they follow their own review chain. Wording corrections that preserve the mailed design are exempt. A wording correction preserves the mailed design when it does not add, remove, or rename any public-facing API; does not change observable behavior or semantics; and does not narrow or eliminate options presented in the mailed revision.

Group boundary. CWG and LWG are specification groups. If CWG or LWG review during the meeting determines that a design change - not a wording correction - is needed, the paper returns to EWG or LEWG. A paper that returns to an evolution group for a design change appears in the next pre-meeting mailing with a new revision number before reaching plenary.

Open question. The interaction between this rule and NB comment resolution during the CD/DIS cycle is an open question for committee discussion. NB comment resolutions sometimes require normative design changes under an external ISO deadline. The committee should consider whether NB comment resolutions require a distinct exception or whether the group-boundary mechanism provides sufficient flexibility.

If you agree with the premise of this paper, I kindly ask you to reach out and be listed as a co-author.

Acknowledgements

Ville Voutilainen, for P2138R4 and for permission to quote private correspondence. Nina Ranns and the co-authors of P3962R0, for documenting the implementers' perspective. Joshua Berne, for identifying the filibuster vulnerability, for the insight that CWG and LWG should not make design decisions (which simplified the mechanism), for raising the NB comment resolution concern, and for the principled distinction between plenary ratification and working-group deliberation. Zhihao Yuan, for finding a loophole in the original exemption that would have swallowed the rule. The delegates who voted in favor of P2138R4 in 2021, who were right.

References

1. **P4003R0** - "Coroutines for I/O" (Vinnie Falco, Mungo Gill, Steve Gerbino, 2026). <https://wg21.link/p4003r0>
2. **P4007R0** - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). <https://wg21.link/p4007r0>
3. **P2583R0** - "Symmetric Transfer and Sender Composition" (Mungo Gill, Vinnie Falco, 2026). <https://wg21.link/p2583r0>
4. **P4100R0** - "The Network Endeavor" (Vinnie Falco, Mungo Gill, 2026). <https://wg21.link/p4100r0>
5. **P4131R0** - "Retrospective: Effects of the WG21 Train Model" (Vinnie Falco, 2026). Post-Croydon mailing.
6. **P4129R1** - "Voting Dynamics" (Vinnie Falco, 2025). Post-Croydon mailing.
7. **P4130R0** - "Appointment Is Policy" (Vinnie Falco, 2026). Post-Croydon mailing.
8. **SD-4** - "WG21 Practices and Procedures." <https://isocpp.org/std/standing-documents/sd-4-wg21-practices-and-procedures>
9. **P2138R4** - "Rules of Design $\Leftrightarrow$ Specification engagement" (Ville Voutilainen, 2021). <https://wg21.link/p2138r4>
10. **P2435R0** - "2021 Summer Library Evolution Poll Outcomes" (Bryce Adelstein Leibach, 2021). <https://wg21.link/p2435r0>
11. **P3962R0** - "Implementation reality of WG21 standardization" (Nina Ranns et al., 2026). <https://wg21.link/p3962r0>
12. **P3941R3** - "Scheduler Affinity" (Dietmar Kühl, 2026). <https://isocpp.org/files/papers/P3941R3.html>
13. **P3941R2** - "Scheduler Affinity" (Dietmar Kühl, 2026). <https://wg21.link/p3941r2>
14. **P3980R1** - "Task's Allocator Use" (Dietmar Kühl, 2026). <https://isocpp.org/files/papers/P3980R1.html>
15. **P3980R0** - "Task's Allocator Use" (Dietmar Kühl, 2026). <https://wg21.link/p3980r0>
16. **P4159R0** - "Make sender_in and receiver_of exposition-only" (2026). <https://isocpp.org/files/papers/P4159R0.html>
17. **P3826R5** - "Fix Sender Algorithm Customization" (Eric Niebler, 2026). <https://isocpp.org/files/papers/P3826R5.html>
18. **P3826R3** - "Fix Sender Algorithm Customization" (Eric Niebler, 2026). <https://wg21.link/p3826r3>
19. **P3927R1** - "task_scheduler Support for Parallel Bulk Execution" (Eric Niebler, 2026). <https://isocpp.org/files/papers/P3927R1.html>

20. **P4154R0** - "Renaming various execution things" (Tim Song, Ruslan Arutyunyan, 2026).
<https://isocpp.org/files/papers/P4154R0.html>
21. **P1000R2** - "C++ IS Schedule" (Herb Sutter, 2018). <https://wg21.link/p1000r2>
22. **P1000R7** - "C++ IS Schedule (proposed)" (Herb Sutter, 2026). <https://wg21.link/p1000r7>